CQRS

separate models for writing and reading

Watch the 4-min video

In one line: CQRS (command query responsibility segregation) uses one model to handle changes and enforce rules, and one or more separate read models shaped for queries, kept up to date from events or change data capture; reads become fast and flexible, at the cost of eventual consistency and more moving parts.

1 One table can't serve both

Your product team wants a search page: filter orders by status, customer name and date, sort by total, with full-text search on the items. But the `orders` table is designed for safe writes: normalised, indexed for lookups by ID, protected by constraints. Adding every search index slows writes, and full-text search is a poor fit for it anyway.

2 The idea

Commands (PlaceOrder, CancelOrder) go to a **write model** that validates them and stores the result. **Queries** go to **read models** built for them: a search index, a denormalised table per screen, a cache, a graph. The read models are updated from the write side, usually through events (lesson 34), the outbox (lesson 37) or change data capture. Each side can use different storage and scale independently.

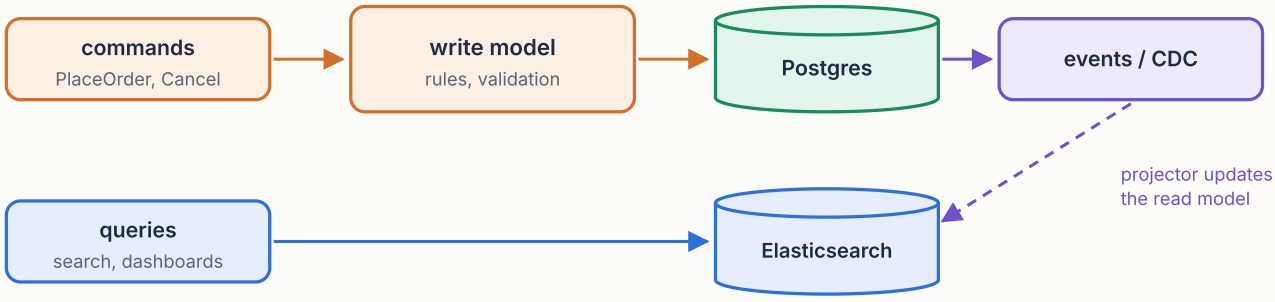


Figure 1. Writes go to Postgres; a projector turns changes into a search-friendly read model.

WORKED EXAMPLE: ORDER WRITES IN POSTGRES, READ VIEWS IN ELASTICSEARCH

The order service handles commands like PlaceOrder and CancelOrder and writes to Postgres in transactions, publishing events through its outbox. A projector consumes `OrderPlaced`, `OrderCancelled` and `CustomerRenamed` and upserts one denormalised document per order into Elasticsearch, with customer name, item text and total. The search page queries Elasticsearch only. When the team needs a new 'orders by region' dashboard, they write a second projector and replay the events to build it.

3 Living with lag

The read model trails the write model by milliseconds to seconds. Right after placing an order, the user's search may not show it. Handle it: update the UI optimistically from the command result, read your own

writes from the write model for a short window, or return the write's version and let the UI wait until the read model has reached it.



hide the lag: optimistic UI, or wait until the projection reaches the write's version

Figure 2. Hide projection lag from the user who just wrote.

4 When it's worth it



Figure 3. CQRS is a tool for specific pressures, not a default.

Projections must be **idempotent** and handle out-of-order or replayed events (versions per document). Keep the ability to **rebuild** a read model from scratch, and monitor projection lag like consumer lag.

COMMON MISTAKES

- Applying CQRS to every simple CRUD screen.
- Updating the database and the search index from the API in two separate steps, without an outbox.
- Projections that can't be rebuilt.
- Ignoring the lag, so users think their action failed.

5 Key terms

CQRS: separate models for commands and queries

Read model / projection: query-optimised copy built from changes

Projection lag: how far the read model trails the writes

Write model: validates commands, enforces rules

Projector: consumer that updates a read model

6 Interview questions

Q1. What is CQRS? BASIC

Separating the model that handles writes (commands and rules) from the models that serve reads, usually with different storage, kept in sync asynchronously.

Q2. Does CQRS require event sourcing? BASIC

No. They fit well together, but CQRS can use a normal database for writes and CDC or an outbox to update the read models.

Q3. Design order search with filters and full-text over a transactional orders database. SCENARIO

Keep orders in Postgres, publish changes via outbox or CDC, project them into Elasticsearch documents, and query only the index; handle lag in the UI.

Q4. Users don't see their new order in search for two seconds. What do you do? SCENARIO

Show it optimistically from the command response, or wait for the projection to reach the write's version; also check projector lag.

Q5. How do you fix a bug in a projection that corrupted the read model? TRICKY

Fix the projector, then rebuild the read model by replaying the events or re-exporting from the write store into a new index, and switch over.

Q6. What's the biggest cost of CQRS? TRICKY

Eventual consistency between models, plus more components to build, run and keep in sync.

Next: SD-I-015 · Transactional Outbox Pattern backendroadmap.com